

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВГУ»)

Факультет прикладной математики, информатики и механики

Кафедра вычислительной математики и информационных
прикладных технологий

Направление 01.04.02 Прикладная математика и информатика

Отчёт

по производственной практике (научно-исследовательская работа)

Тема	Разработка аппаратной реализации 2D графического ускорителя
Отчетный семестр	4

Обучающийся	2 курс, 11 группа, Старухин Д.М.
Руководитель от кафедры	к.ф.-м.н., доц. Медведева О.А.

Воронеж 2026

Содержание

Введение	3
Постановка задачи	4
Актуальность поставленной задачи	5
1. Обзор архитектур графических процессоров	6
1.1. Обзор архитектур графических процессоров компании Nvidia	6
1.2. Обзор архитектур графических ускорителей компании AMD	12
1.3. Заключение обзора	20
2. Архитектура ядра	23
2.1. Различие и сходство архитектур ядер CPU и GPU	23
2.2. Интерфейсы графического ядра	24
2.2.1. Интерфейс передачи данных и инструкций (AHB)	26
2.2.2. Интерфейс передачи данных с регистровой картой (APB)	26
2.3. Поток	27
2.3.1. Классификация параллельных вычислительных моделей (SIMT vs SIMD, MIMD)	28
2.3.2. Структура потока	29
2.3.2.1. Арифметически-логическое устройство	30
2.3.2.2. Математический сопроцессор	31
2.3.2.3. Регистровый файл	33
2.3.2.4. Архитектура системы	35
2.3.2.4.1. Архитектура набора инструкций RISC-V	36
2.3.2.4.2. Архитектура набора инструкций графического ядра	38
2.3.2.5. Общая структура потока	39
2.3.3. Структура ядра	40
Заключение	41
Литература	42

Введение

В этой работе будет представлена аппаратная архитектура реализации ядра для графических ускорителей. Также, в этой работе будут рассмотрены другие архитектуры графического ядра у ведущих компаний в разработке графических ускорителей. С учётом уже существующих архитектур графических ядер, в работе будет представлена собственная архитектура графического ядра.

Постановка задачи

Компанией АО «ПКК» Миландр была поставлена задача разработать аппаратную реализацию 2D графического ускорителя. Перед началом аппаратной разработки необходимо описать архитектуру, которой должен соответствовать сложно-функциональный (СФ) блок.

При разработке архитектуры учитываются проделанные ранее обзор и анализ 2D графических ускорителей [1]. Разработка ведется с учётом предложенной ранее [2] архитектурой графического ускорителя.

Перед тем как разработать аппаратную реализацию 2D графического ускорителя, необходимо и важно разобраться в предназначении и структуре различных компонентов IP-блока. Одним из таких компонентов, является графическое ядро.

Графическое ядро имеет довольно сложную архитектуру, которое состоит из элементов, которые должны корректно соответствовать друг другу. Поэтому, перед разработкой графического ядра, нужно разработать его архитектуру, с учётом разработок графических ядер других кампаний.

Актуальность поставленной задачи

В современном мире, в условиях развития нейросетей и импортозамещения, создание собственных специализированных графических и вычислительных архитектур является довольно важной задачей.

Нейросети все больше вовлекаются во многие аспекты, начиная от развлекательной индустрии, заканчивая научными исследованиями. Искусственный интеллект образовал огромный спрос видеокарт, с помощью которых можно собрать вычислительный сервер, на которых, можно развернуть нейросеть для решения различных задач. Видеокарта состоит из большого количества элементов, одним из которых является графический процессор [3]. графический процессор обладает большим количеством ядер, которые выполняют огромный объем вычислений с плавающей точкой.

Помимо искусственного интеллекта, графические ядра также могут использоваться в вычислениях рендеринга графики, обработки видео или фото, а также в различных научных вычислениях, где требуется рассчитывать тысячи параллельных математических вычислений.

Разработка архитектуры графического ядра позволит снизить зависимость от зарубежных архитектур, таких, как AMD и Nvidia. Благодаря разработке IP-блока, можно достичь создания доверенных систем, которые не будут содержать в себе наличия аппаратных «закладок» и недокументированных функций в критически важной инфраструктуре.

1. Обзор архитектур графических процессоров

Перед тем, как заниматься разработкой архитектуры графического ядра, стоит обратить внимание на уже существующие разработки различных компаний.

1.1. Обзор архитектур графических процессоров компании

Nvidia

Одна из компаний, занимающаяся разработкой графических ускорителей является Nvidia, которая с 1993 года занимается разработкой аппаратных ускорителей вычисления [4]. Nvidia за более 30 лет разработала большое количество микроархитектур для графических процессоров.

Одной из первых архитектур, разработанных Nvidia являлась архитектура Rankie, которая использовалась с 2003 по 2005 года в графических процессорах. Одним из графических процессоров, разработанным на микроархитектуре Rankie являлся NVIDIA NV30. Он обладал 125 миллионами транзисторами с техпроцессом 130нм. Графический процессор на тот момент обладала 4 пиксельными шейдерами, 3 вершинных шейдера, 8 блоков наложения текстур и 4 блоками вывода рендеринга (ROP (Render Output Unit)) [5].

Блок диаграмма архитектуры графического процессора NVIDIA NV30 представлена на рис. 1:

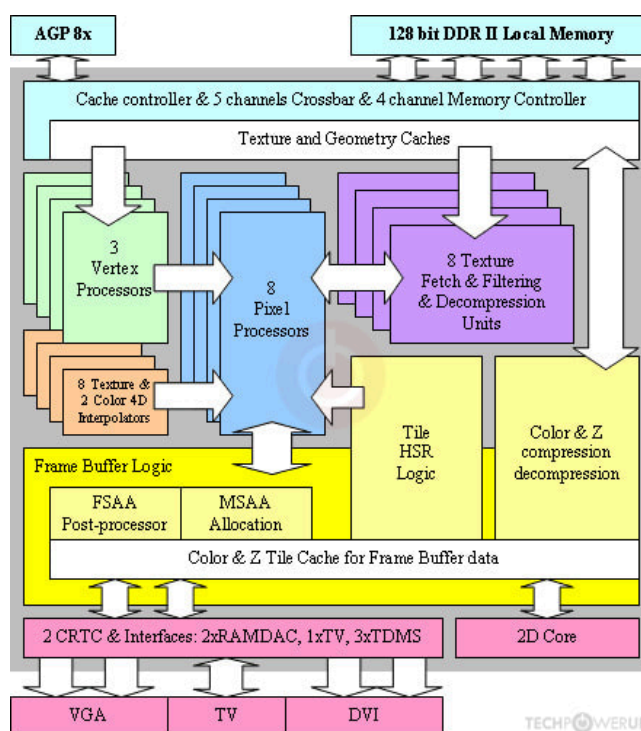


Рис. 1 — Блок схема архитектуры графического процессора NVIDIA NV30

По рис. 1 заметно, что архитектура очень похожа на современные 2D-графические ускорители, разрабатываемые в микроэлектронике. Понятие графического ядра тогда не существовало, вместо графического ядра все задачи выполняли вершинные и пиксельные процессоры. До 2006 года видеокарты использовали жестко разделенные процессоры, которые выполняли специализированные задачи. Но на сегодняшний день, вершинным и пиксельным процессорам пришли на замену унифицированные шейдерные ядра. Такие ядра выдавали более высокую производительность чем шейдеры, в последствии чего, будущие архитектуры графических процессоров стали основываться на использовании большого количества универсальных блоков. Такие блоки стали называться потоковыми процессорами (SP) [6].

Первая архитектура, которая стала использовать SP, как основу в графических процессорах стала архитектура Tesla, представленная с 2006 года. Данная архитектура в первую очередь отличалась от архитектуры Rarke использованием SP блоков, что являлось в новинку в архитектурах графических процессоров компании Nvidia. Архитектура графического процессора изображена на рис. 2:

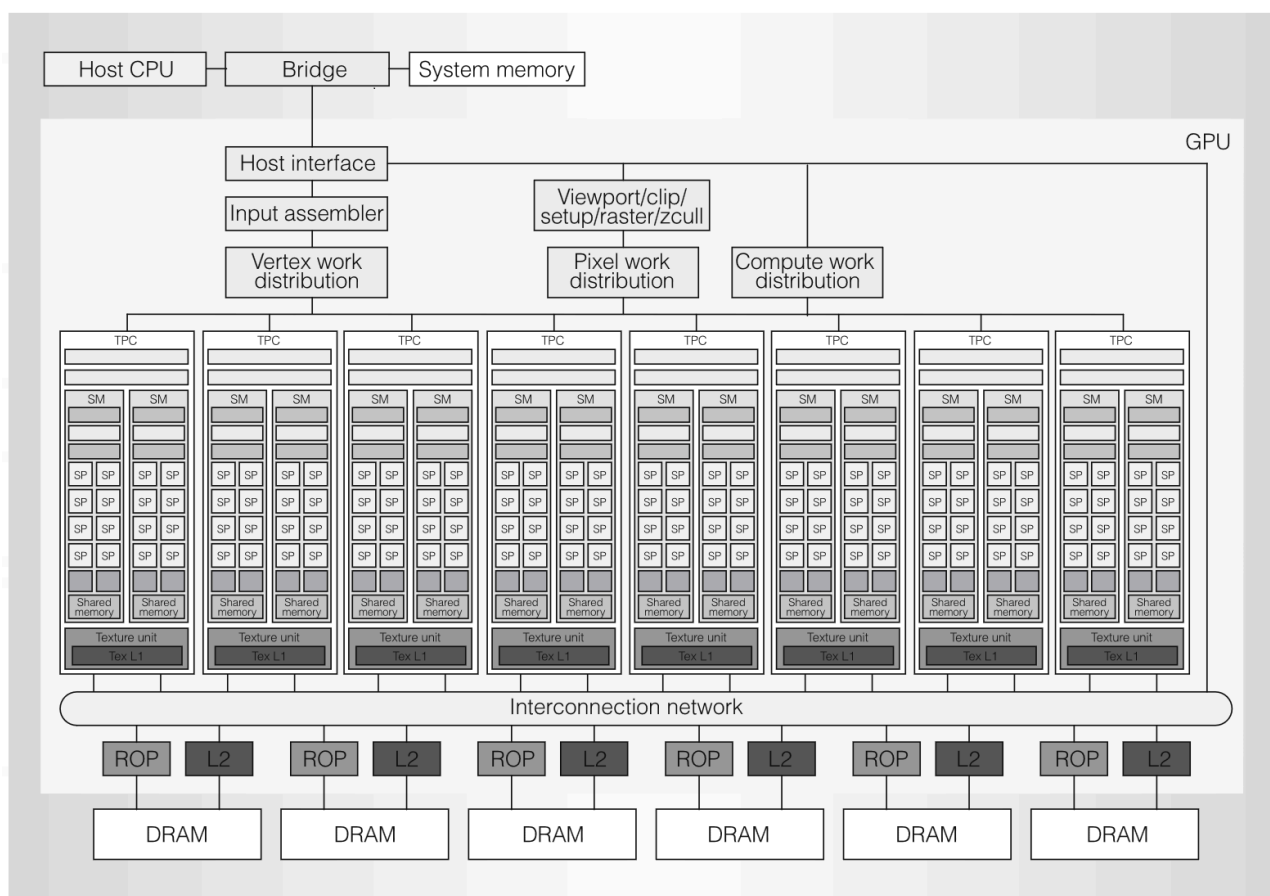


Рис. 2 — Архитектура графического процессора Tesla

Можно заметить, что архитектура Tesla сильно различается от Ranki. Теперь всю работу выполняют кластеры обработки текстур (TPC - texture/processor cluster). В графическом процессоре находится множество TPC и каждый из них является комплексным компонентом в архитектуре. Компонент TPC, в свою очередь, состоит из геометрического контроллера (Geometric controller), текстурного блока (Texture unit) контроллера потокового мультимикропроцессора (SMC) и самих потоковых мультимикропроцессоров (SM). Каждый потоковый мультимикропроцессор представляет собой набор из кешей (I cache и C cache), обработчика прерываний (MT issue), Блок специальных функций (SFU), общая память и потокового процессора (SP) [7]. Архитектура TPC представлена на рис. 3:

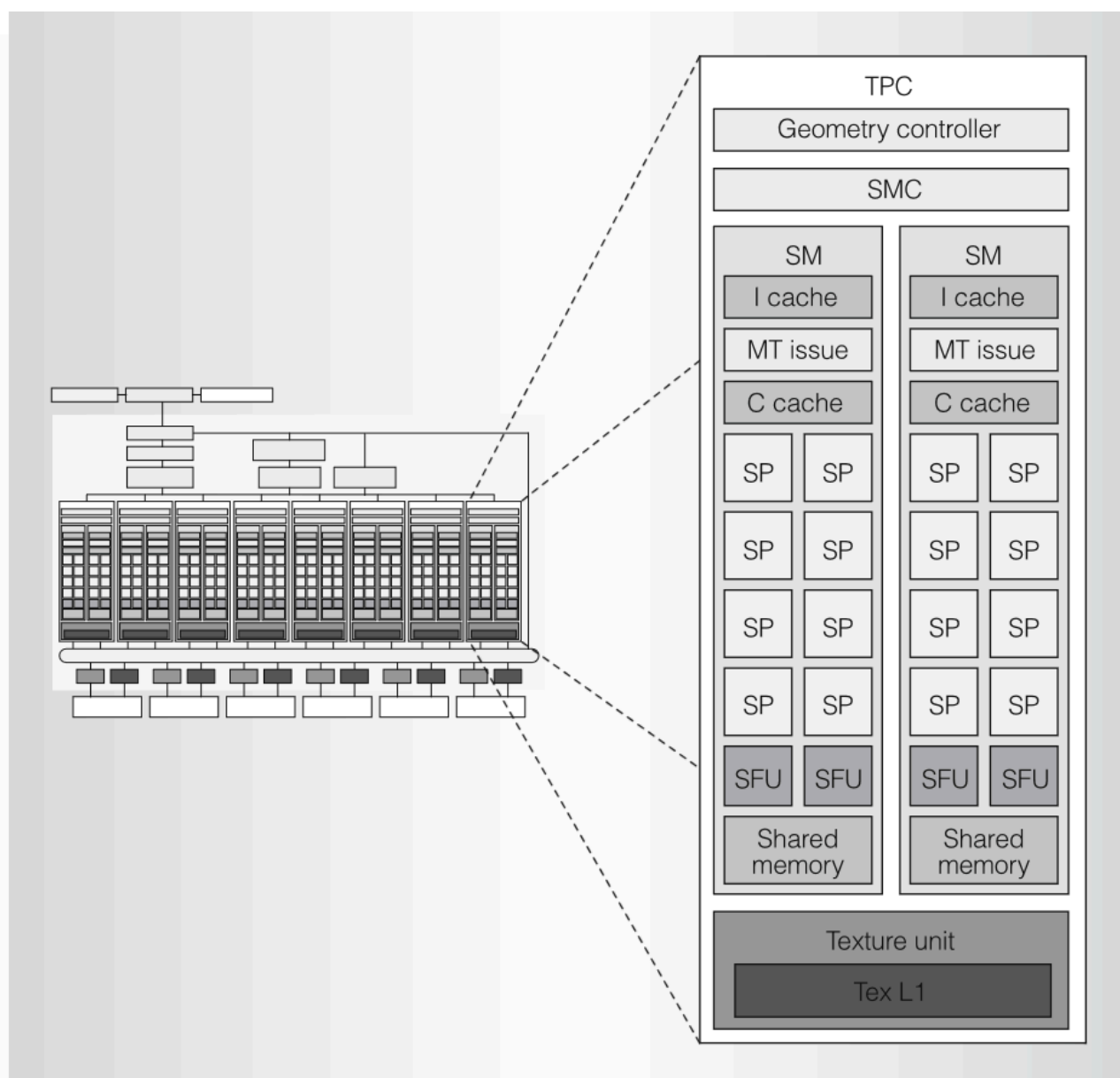


Рис. 3 — Архитектура графического процессора Tesla

По своей сути, основную работу с вычислениями выполняли SP, все сложные функции и операции, такие как вычисления синуса, косинуса или экспоненты отдавались на вычисление блоку SFU. В следующих архитектурах компании Nvidia, SP заменяются на CUDA-ядра, которые будут являться архитектурой параллельных вычислений и основой для разработки графических процессоров. Архитектура, в которой появились CUDA-ядра является архитектурой Fermi, используемая в видеокартах, начиная с 2010 года. Одной из главных особенностей стало появление CUDA-ядер. На рис. 4 можно увидеть компонент SM в архитектуре Fermi:

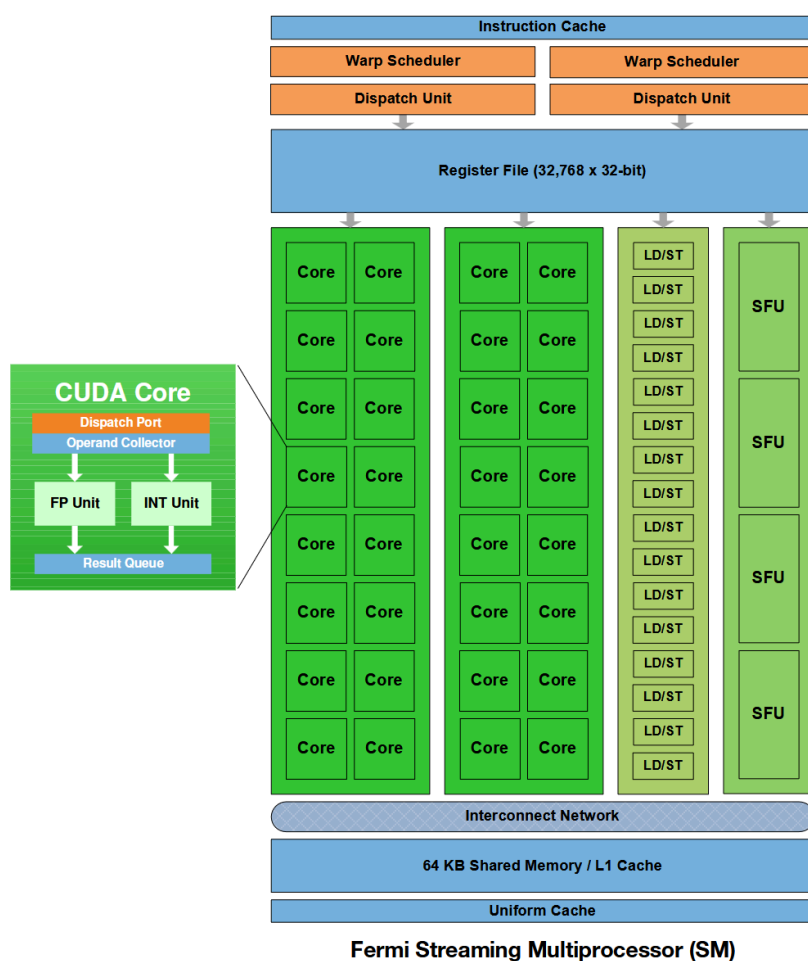


Рис. 4 — Поточковый мультипроцессор архитектуры Fermi (SM Fermi)

В потоковом мультипроцессоре на место SP появились CUDA-ядра. CUDA-ядро состоит из порта для блока распределения и выдачи задач, операнда-коллектора, вычислительных элементов и очереди для результатов выполнения поставленных задач ядру. Вычислительные элементы являются FP-вычислительные блоки (FPU), для вычисления чисел с плавающей точкой и INT-вычислительные блоки (ALU), для вычислений целочисленных чисел. Также в потоке мультипроцессора появился блок LD/ST (Load/Store), который является блоком загрузки и сохранения. Также в SM появился реги-

стовый файл (Register File), для сохранения промежуточных значений, планировщик (Warp Scheduler) и блок выдачи и распределения задач (Dispatch Unit) [8].

Теперь рассмотрим более современную микроархитектуру - Turing, разработанную Nvidia для видеокарт, начиная с 2018 года. Данная архитектура представляла собой первую микроархитектуру, которая поддерживала трассировку лучей в реальном времени и применялась в потребительских продуктах [9]. Одним из графических процессоров, разработанных на микроархитектуре Turing являлся графический процессор NVIDIA TU102. С его архитектурой можно ознакомиться на рис. 5:



Рис. 5 — Архитектура графического процессора NVIDIA TU102

Такая архитектура является уже сильно улучшенной по сравнению с предыдущими представленными. За 8 лет, количество графических ядер у графических процессоров заметно увеличилось. На архитектуре графического процессора NVIDIA TU102 можно увидеть около 4608 универсальных ядер (CUDA-ядер), 72 ядер для выполнения рей-трейсинга и 576 тензорных ядер. Один графический процессор может

иметь 6 вычислительных графических кластеров (GPC), в каждом из которых по 12 потоковых мультипроцессоров (SM). В каждом мультипроцессоре есть 1 ядро для выполнения рей-трейсинга и 4 вычислительных единицы, в каждом из которых есть по 16 вычислительных CUDA-ядер и 2 тензорных ядра. В новых потоковых мультипроцессорах помимо CUDA-ядер появились ядра для рей-трейсинга (RT Core) и тензорные ядра (Tensor Core) для умножения и свёртки матриц [10]. Структуру потокового мультипроцессора (SM) Turing NVIDIA TU102 можно увидеть на рис. 6:

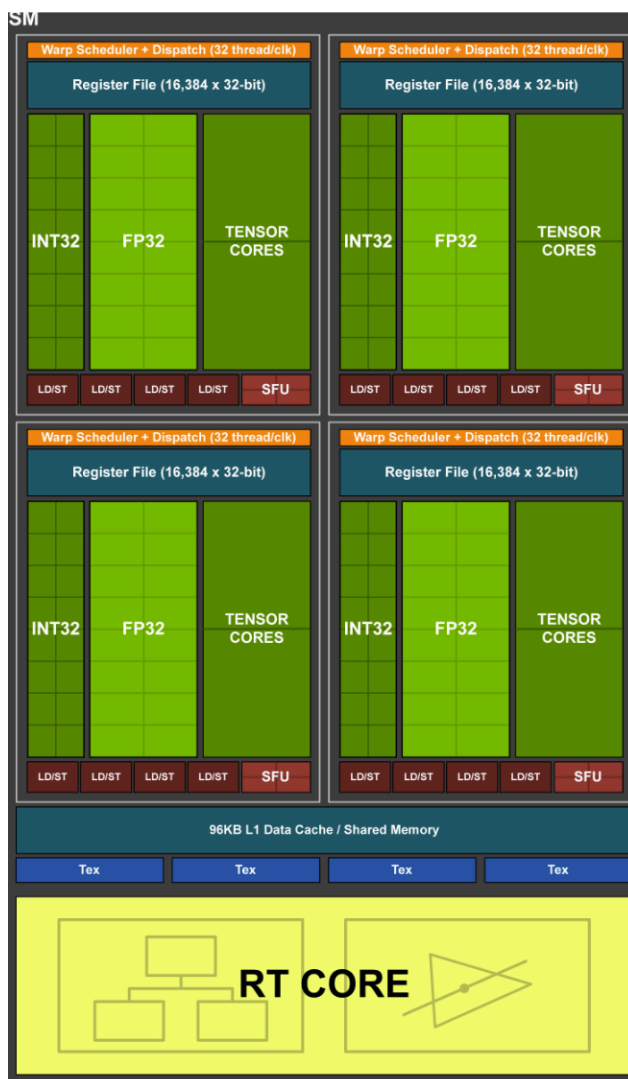


Рис. 6 — Архитектура SM NVIDIA TU102

Количество LD/ST-блоков знатно меньше, чем вычислительных элементов у потокового мультипроцессора. Если в архитектуре Fermi использовалось на 2 ядра 1 LD/ST блок, то количество ядер на 1 блок загрузки и сохранения увеличилось вдвое. В будущих архитектурах, структура потокового мультипроцессора в общем останется неизменной. В архитектуре Blackwell заметно общее увеличение SM-блоков и улучшение тензорных и рей-трейсинговых ядер.

1.2. Обзор архитектур графических ускорителей компании AMD

Компания Advanced Micro Devices, Inc. или же AMD является американским производителем интегральных микросхем и электроники. Данная компания в том числе занимается не только разработкой центральных процессоров, но и графических. Компания была основана в 1969 году, но первый графический процессор под брендом AMD стал ATI Radeon серии R100, выпущенный в 2000 году. По своей сути, графическими процессорами на тот момент занималась компания ATI, которая в будущем, в последствии была выкуплена самой AMD [11].

ATI занималась разработкой видеокарт для вычислительных машин, начиная с конца 80-ых, когда использовались фиксированные функции для вычислений графики. Не было шейдеров, вычисления происходили быстро, но при этом крайне ограничено. Видеокарты были довольно ограниченными в плане использования, ведь не допускали написания собственного кода, а лишь позволяли настраивать предустановленные этапы. AMD описывает эту эпоху, как Fixed Function (Фиксированные функции).

На смену первой эпохе пришла эпоха Simple Shaders или же эпоха простых шейдеров. Появились программируемые шейдеры. Это позволяло писать программистам различные алгоритмы и программы, расширяя возможности графического процессора [12]. Шейдеры разделялись на вершинные и пиксельные. В такой эпохе могла возникнуть проблема, когда одни шейдеры простаивали, пока шейдеры другого типа были перегруженными. Данная эпоха давала гибкость, но при этом была не так эффективной, как следующая эпоха.

Третья эпоха называется Graphics Parallel Core или же параллельные графические ядра. Главная особенность этой эпохи заключается в унифицированной архитектуре. На смену векторным и пиксельным шейдерам приходят ядра, способные выполнять большой список задач, повышая гибкость графического процессора и уменьшая простой элементов от специализации блоков.

Все три эпохи изображены на рис. 7:

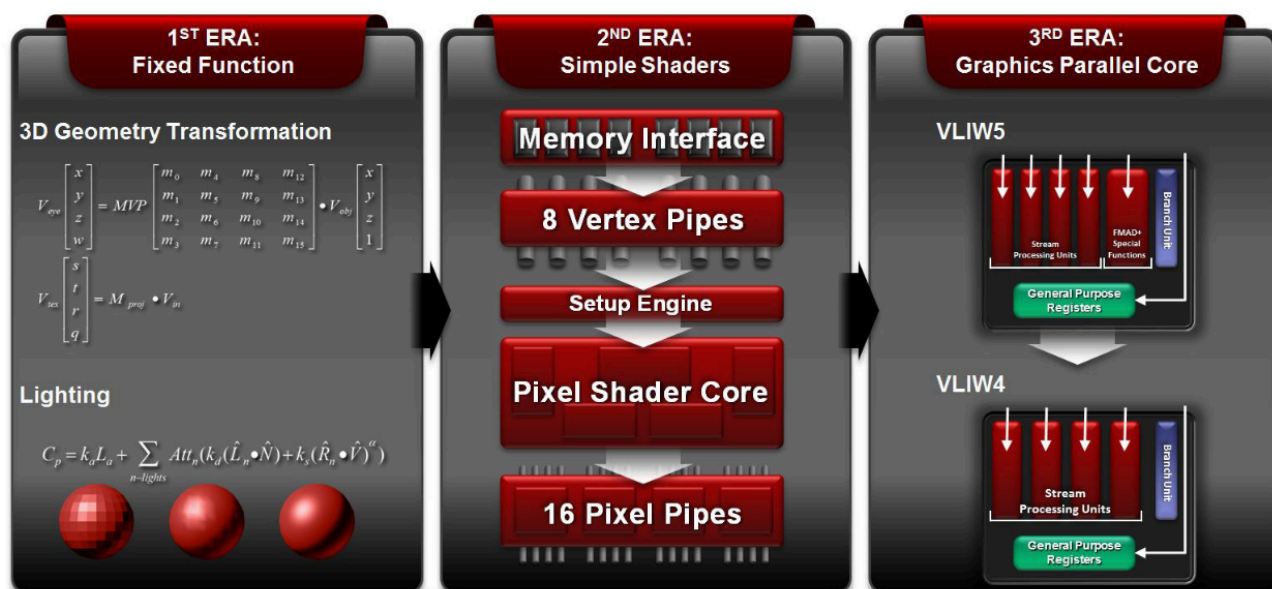


Рис. 7 — Эволюция видеокарт по классификации AMD

Рассмотрим архитектуру AMD TeraScale, которая использовалась с 2005 по 2013 год. Первый графический чип, разработанный на архитектуре TeraScale назывался R600, а все графические процессоры, основанные на R600 относились к семейству R600 (R600-Family) [13]. Архитектура семейства R600 показана на рис. 8

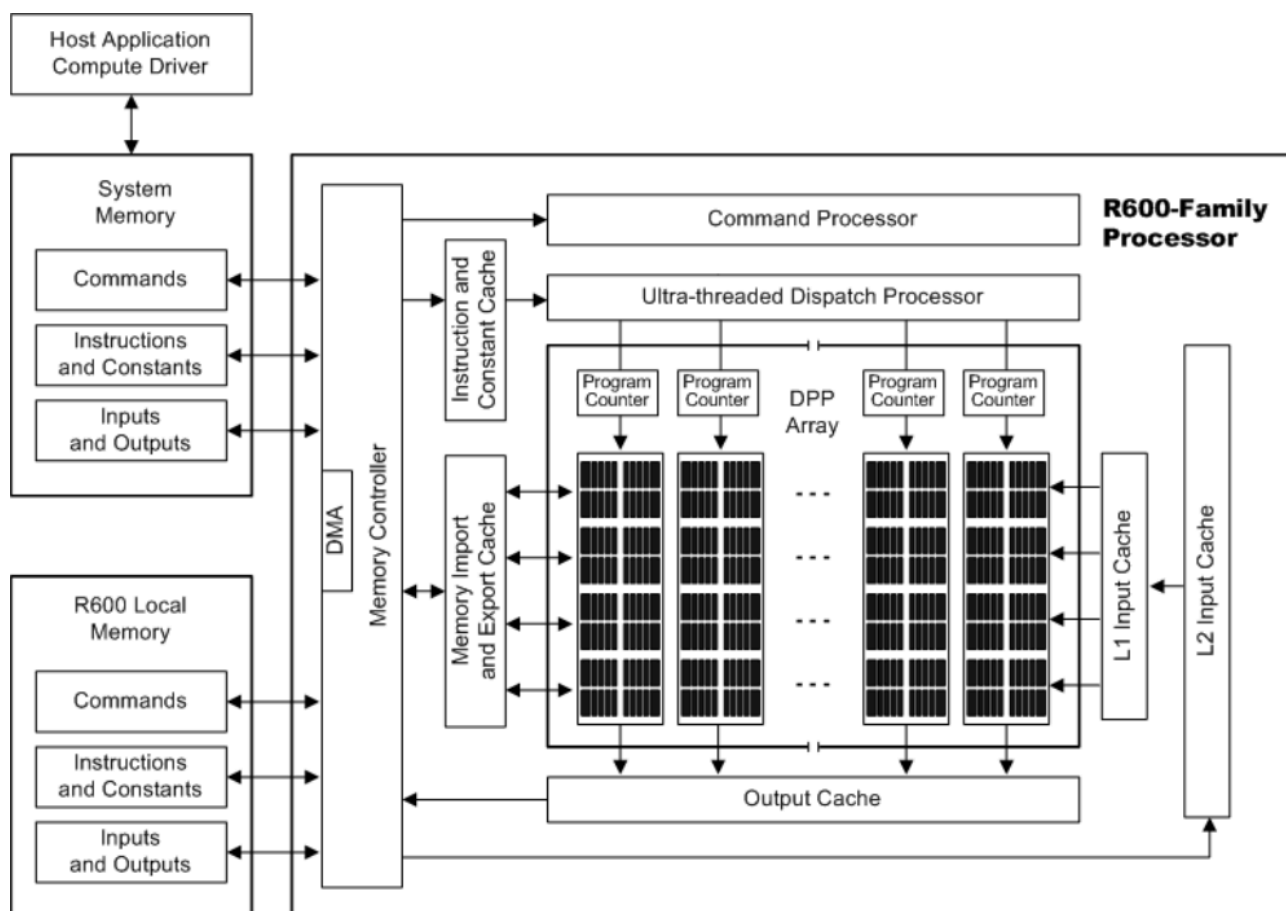


Рис. 8 — Графический процессор семейства R600

Можно увидеть, что графический процессор семейства R600 состоит из большого количества кешей, процессора команд, контроллера памяти, блока выдачи задач потокам и массива параллельных процессов (DPP - Data Parallel Processor). DPP состоит из набора SIMD-конвейеров, каждый из которых независим от других. SIMD-конвейер является набором потоков, которые выполняют вычисления над данными.

R600 семейство было первым поколением, которое применила унифицированную шейдерную архитектуру. Вместо использования отдельных вершинных и пиксельных шейдеров, графический процессор содержал в себе универсальные ядра, способные заменить специализированные шейдеры [14].

Дальше архитектуру TeraScale стали модернизировать и структура семейства процессоров Evergreen стала выглядеть иначе, чем семейство R600. Ознакомится с структурой процессора семейства Evergreen, можно увидеть на рис. 9:

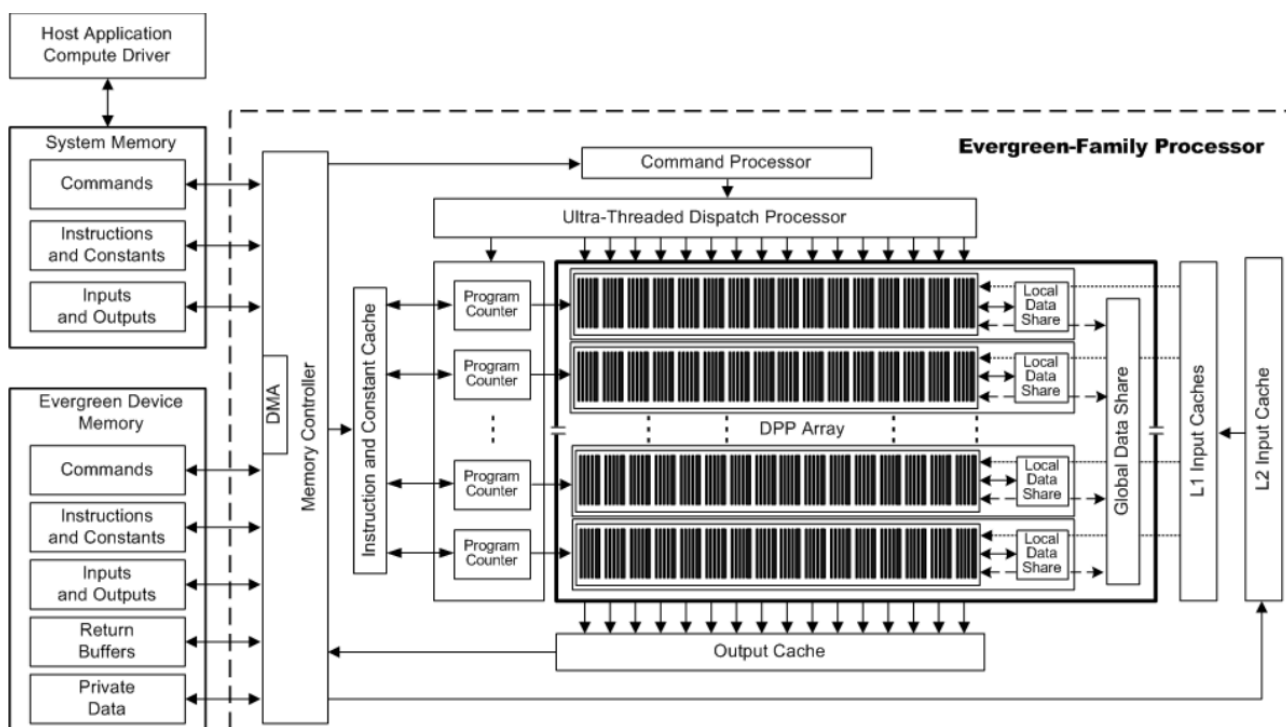


Рис. 9 — Графический процессор семейства Evergreen

Можно заметить, что у семейства графических процессоров Evergreen появились дополнительные компоненты, которые войдут в будущие архитектуры графических процессоров AMD. Одним из таких является локальная разделяемая память или Local Data Share (LDS). LDS предоставляет быструю память для обмена данными между потоками в SIMD-конвейере. При этом, также есть и память общего назначения (GDS), которая предоставляет общую память для всех вычислительных элементов в класте.

ре. Иерархию разделяемой памяти в потоковых процессорах архитектуры семейства Evergreen можно увидеть на рис. 10:

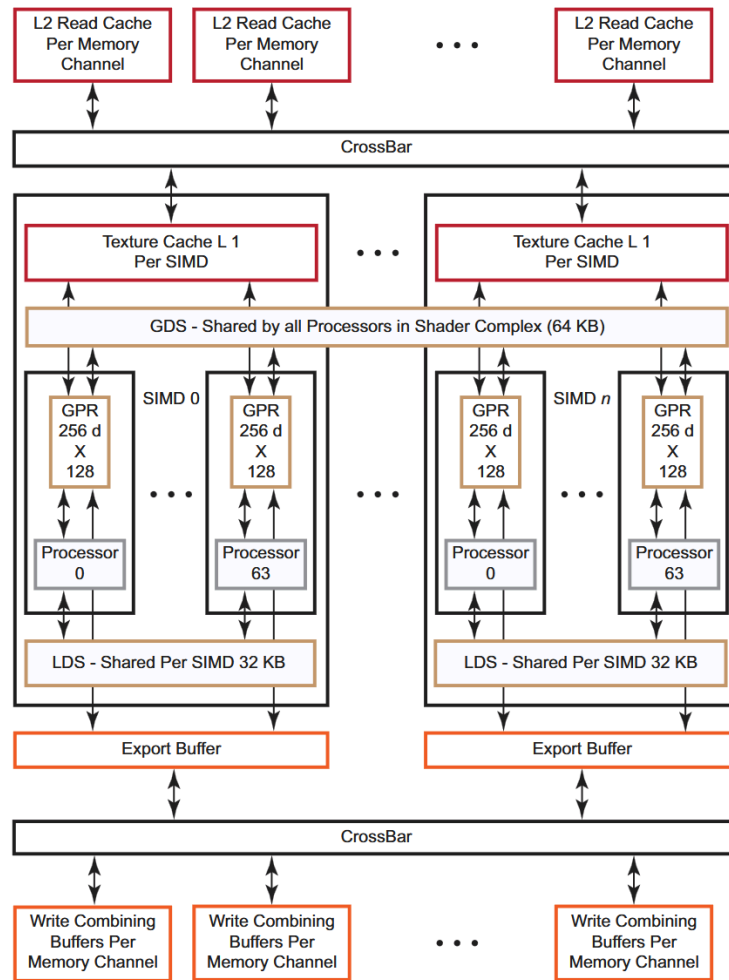


Рис. 10 — Иерархия разделяемой памяти в семействе потоковых процессоров Evergreen

Также в потоке появится сущность под названием GPR - General Purpose Registers или регистры общего назначения. Они предназначены для хранения промежуточных данных, которые используются потоками прямо во время вычислений. Главная особенность GPR заключается в сверхбыстром доступе к индивидуальным данным для каждого потока. Также благодаря GPR допускается сохранение обеспечения параллелизма в графическом ядре, так как у каждого потока есть свой блок GPR [15].

В документации архитектуры Evergreen появились следующие термины:

- Рабочий элемент (Work-item) - набор параллельно выполняемых потоков на устройстве, вызванных при

помощи команды. Рабочий элемент не зависит от другого рабочего элемента и может исполнять параллельно код с остальными. Если ему необходимо получить

данные используемые или уже обработанные любым другим рабочим элементом, он может получить их через общую память. По своей сути рабочий элемент это поток;

- Рабочая группа (Work-group) - набор взаимодействующих рабочих элементов, исполняющихся на одном

устройстве. У каждой рабочей группы есть своя рабочая память. По своей сути это и является вычислительным блоком в аппаратном виде [16]. В свою же очередь Рабочие группы организуются в волновой фронт или же в Wavefront (Warp) [17]. Количество потоков Wavefront определяют его размер. Обычно этот размер либо 32, либо 64. Все потоки, находящиеся в одном Wavefront выполняют одну и ту же инструкцию над разными потоками данных.

В архитектурах начиная с Vega, work-group имеют другую терминологию. Рабочая группа становится набором Wavefront'ом, которые имеют возможность быстро синхронизироваться между собой. Wavefront'ы также могут передавать друг другу данные через общий LDS.

После архитектуры TeraScale AMD стала выпускать графические процессоры на архитектуре GCN - Graphics Core Next, что в переводе называется «Графическое ядро следующего поколения». Данная архитектура использовалась в графических ускорителях с 2012 по 2020 год.

Одной из главных особенностей GCN - это отказ от VLIW (very long instruction word) - очень длинных машинных команд. Раньше TeraScale получала на вход команды, в которых содержалось несколько последовательных инструкций, которые нужно выполнить элементу DPP. С переходом на новую архитектуру графических процессоров, AMD решило отказаться от VLIW в пользу скалярной архитектуры набора команд (GCN Quad). Это позволило повысить производительность, за счёт более эффективного использования ресурсов, что привело и к снижению потребления. При этом, так как вычислительные компоненты графического ядра получали лишь одну инструкцию, а не VLIW, это повысило гибкость методов компиляции, которые упрощают разработку средств отладки и анализа. Для VLIW приходилось прибегать к аккуратной оптимизации, с которой можно было достичь пика производительности графического ядра. При этом, возникали конфликты с регистрами при использовании VLIW, которые необходимо было решать. GCN Quad архитектура инструкций лишала вышеперечисленных проблем и поэтому она пришла на замену VLIW [18].

Второй главной особенностью архитектуры GCN - переход из SIMD-конвейеров в вычислительные блоки (CU - Compute Unit). В одном вычислительном блоке находилось:

- sALU (Scalar ALU) и причастный к нему скалярный GPR. sALU выполняет скалярные вычисления. sALU

обрабатывает данные для всего Wavefront'а целиком, а не для отдельных потоков.

Блок выполняет вычисления индексов, ветвления и uniform операций;

- 4 SIMD-конвейера vALU (Vector ALU) и к ней причастной GPR. Выполняет векторные операции над

потоками. Каждый vALU содержит в себе 16 процессорных элементов (PE);

- Локальная память LDS;
- Доступ чтения и записи к векторной памяти кеша 1-ого уровня [19];
- Кеш инструкций и констант, которые разделены для 4-ёх CU;
- Декодировщик, захватчик команд, буфер и обработчик прерываний.

Структуру вычислительного устройства можно увидеть на рис. 11:

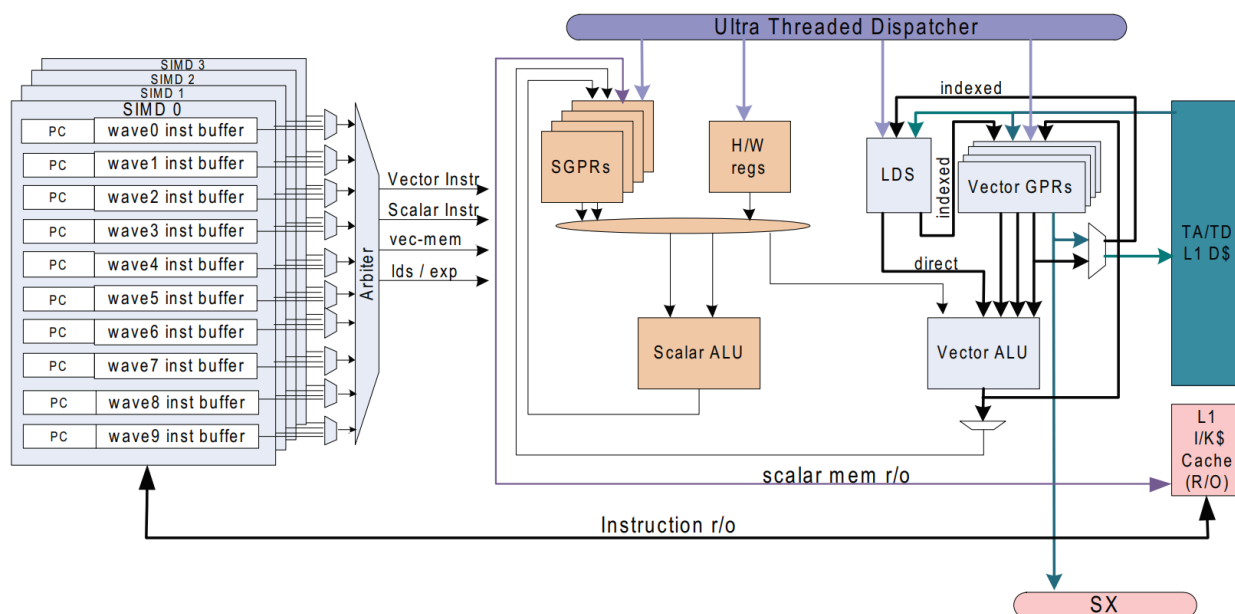


Рис. 11 — структура вычислительного блока GCN

Архитектурой GCN обладает серия графических процессоров Southern Islands (Southern Islands Series Processor, видеокарты этой серии также называют SI-GPU) Общая структура архитектуры графических процессоров серии Southern Islands показана на рис. 12:

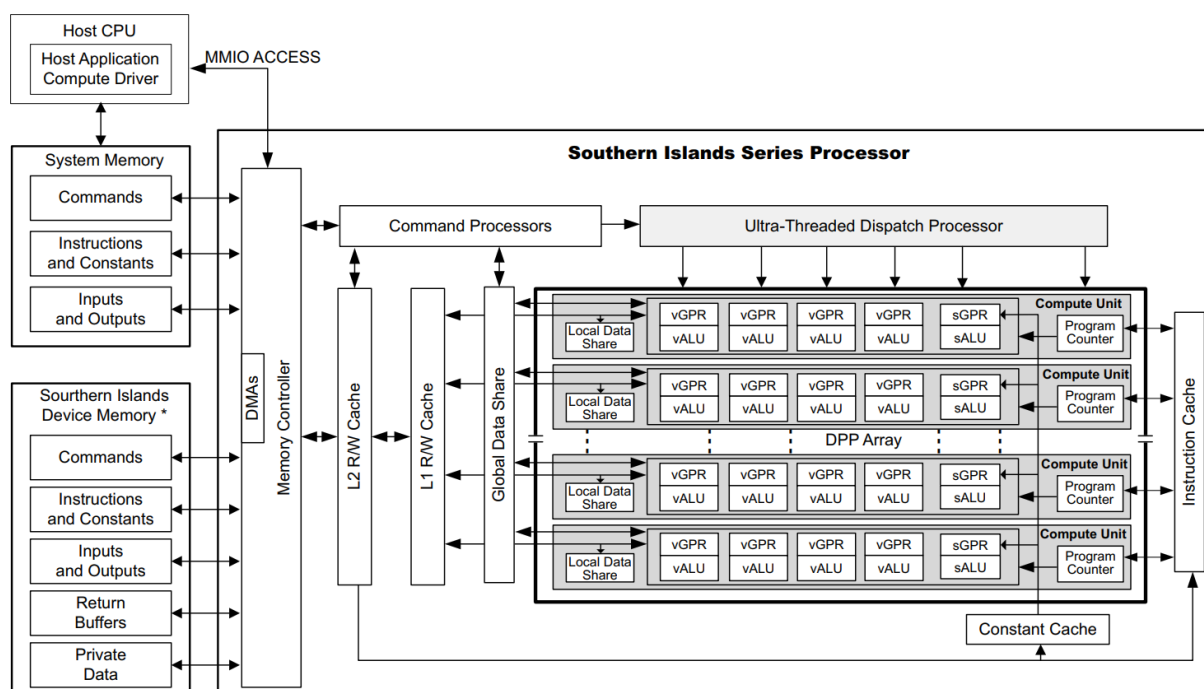


Рис. 12 — Архитектура графических ускорителей серии Southern Islands

На смену GCN, с 2019 года, приходит архитектура RDNA (Radeon DNA). RDNA обладала новой сущностью, называемой WGP (Work Group Processor) или процессор рабочей группы. WGP первой архитектуры RDNA состоит из 2 CU. В GCN поддерживался размер волнового фронта, равный 64 рабочим элементам. Архитектура RDNA поддерживает как 32-поточные, так и 64-поточные архитектуры с различными размерами Wavefront'a [20]. Количество вычислительных элементов vALU увеличено до 32, из-за чего вычисления проводились за 1 такт, вместо 4 тактов, которые наблюдались в архитектуре GCN. При этом, число вычислительных элементов на 1 CU остается неизменным.

Также иерархия кешей усложняется в архитектуре RDNA в пользу ускорения работы вычислительных блоков с памятью. С появлением WGP появляется кеш инструкций и данных 0-ого уровня, который выделен для каждого ядра. Кеш 1 уровня становится общим. Разницу архитектур GCN и RDNA можно увидеть на рис. 13:

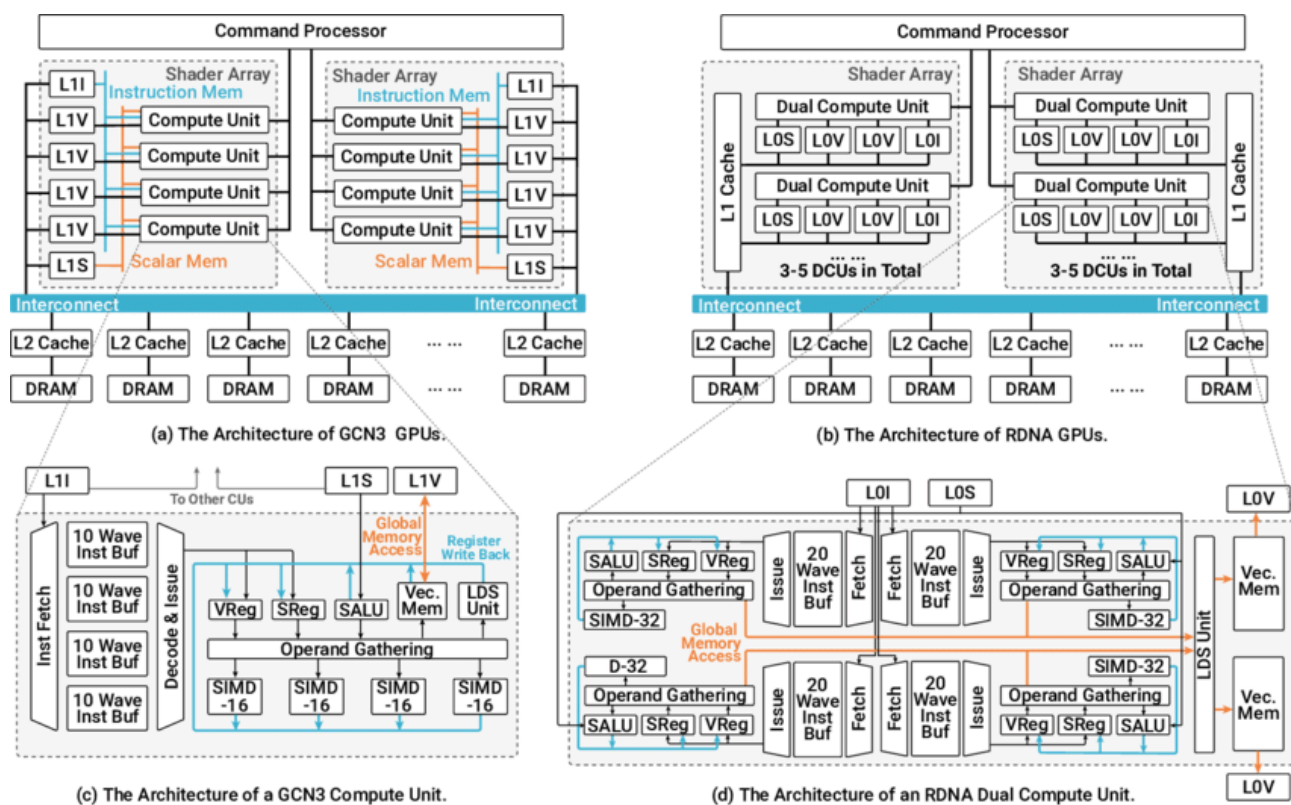


Рис. 13 — Сравнение архитектур графических ускорителей GCN и RDNA

Архитектура RDNA изображена на :

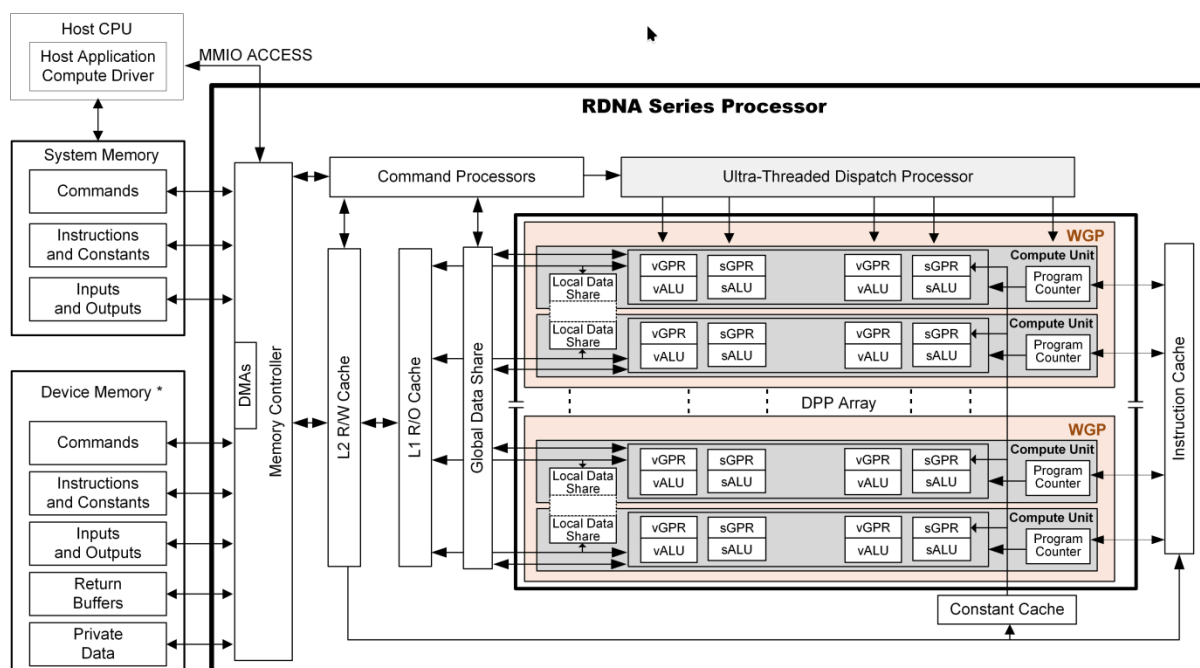


Рис. 14 — Архитектура графического процессора RDNA

В RDNA SGPR хранит в себе различные данные состояний ядра, промежуточные значения и управления потоками. Некоторые из сигналов, хранящиеся в SGPR выводятся напрямую к необходимым блокам, минуя интерфейс чтения данных. Потоки с помощью SGPR можно выключать, образуя возможность создавать шейдерами с

блоками условий (создание ветвей). Также, к примеру, выводится напрямую результат работы сравнения для обеспечения скорости отработки операций и чтения результатов сравнения.

При этом архитектура RDNA обеспечивает обратную совместимость с архитектурой GCN и поддерживает 7 базовых типов инструкций:

- скалярные вычисления;
- скалярный доступ к памяти;
- векторные вычисления;
- векторный доступ к памяти;
- инструкции ветвления (ветви);
- команды экспорта данных;
- служебные сообщения.

1.3. Заключение обзора

Благодаря рассмотрению различных архитектур известных компаний, производящих графические процессоры, были выявлены ключевые особенности при построении графического ядра. Ведущие производители графических процессоров имеют общие концепции построения архитектур графических процессоров, но при этом каждая из архитектур имеет свои тонкости реализации.

Все графические процессоры основаны на большом количестве вычислительных блоков, способных вычислять огромный пласт данных, параллельно. Архитектура в построении работы вычислительных блоков основана на SIMD, потому что зачастую работа с данными подразумевает использование одних и тех же инструкций. Вместо использования больших и длинных команд, содержащих в себе набор инструкций, лучше использовать атомарные инструкции для более упрощенного взаимодействия с ядрами графического процессора.

AMD использует структуру ядра, где существуют универсальные арифметически-логические устройства (ALU), способные производить операции с плавающей точкой. AMD подразделяет 4 ALU в ядре на поток под работу с различными данными и 1 ALU под работу с данными, которые нужны всем потокам в ядре. Nvidia дробит ALU на два типа: для работы с целочисленными данными и для работы с вещественными данными, что сепарирует работу с адресацией и вычислением вещественных значений, требуемых для вычисления графики.

Ядра Nvidia обладают общим местом для хранения промежуточных и константных данных, называемый Регистровым файлом. Ядра AMD обладают локальной разделенной памятью, предоставляя всем потокам доступ к ней, в то время, как каждый поток ядра обладает собственными регистрами для сохранения константных и промежуточных значений. Оба производителя графических процессоров используют кеши для хранения данных, создавая некую иерархичность памяти. Целью использования кешей вызвано ускорением работы с памятью и уменьшением простоя в прочтении данных с памяти, что крайне важно для достижения пиковой производительности видеокарты.

Обе компании стремятся к масштабируемости - последние архитектуры компаний не ограничиваются конкретным числом кластеров, все решения можно масштабировать, конфигурируя продукт под поставленные требования.

Все архитектуры являются конвейерными, чтобы обеспечить производительность и уменьшить простой отдельных элементов ядра.

На основе архитектур графических процессоров можно составить собственную первую итерацию графического ядра.

2. Архитектура ядра

В этом разделе описана разработка архитектуры графического ядра. Каждый отдельный компонент будет разобран до атомарной логики с подробным описанием механизма его работы. Постепенно описывая каждый элемент и связуя их между собой, в конце концов появится общая концепция архитектуры графического ядра. Разработка архитектуры ядра будет основана на обзоре существующих архитектур, а также знаний приобретенных из книг по разработке процессорных архитектур и цифровой схемотехники.

Некоторые наработки и принципы можно взять также из идей и наработок процессорных ядер, но для этого нужно различить структуру между архитектурами ядер CPU и GPU

2.1. Различие и сходство архитектур ядер CPU и GPU

Современные графические ускорители выполняют большое количество мелких задач одновременно. В отличие от центрального процессора (CPU), который предназначен для последовательного выполнения комплексных задач, графические ускорители спроектированы так, чтобы те выполняли множество небольших и независимых вычислений, таких как, к примеру отрисовки пикселей или обучение нейросетей. Одну такую мелкую задачу может выполнять специализированный блок, который называют ядром графического ускорителя. В современных видеокартах (GPU), количество ядер может скалироваться от нескольких тысяч до нескольких десятков тысяч.

Различие количества ядер у CPU и GPU является в том, что центральный процессор выполняет большой спектр различных, сложных и разнообразных задач, последовательно, в то время, как ядро GPU является узкоспециализированным ядром, который может выполнять лишь определенный список задач, которые приходят на обработку. Объем задач у CPU и GPU сильно отличается, ядро CPU обладает большим размером кеша, чем ядро GPU. Чем меньше функционал аппаратного блока и размер хранимых локальных данных, тем меньше размер аппаратного блока. Следовательно, так как CPU обладает намного большим размером функционала одного ядра, он обычно намного больше ядра GPU.

В отличие от ядер CPU, ядра GPU выполняют параллельно между собой похожие операции над данными. В компьютерной графике, мультимедиа, криптографии и машинном обучении часто происходит ситуация, когда нужно работать с большим объ-

емом данных, которым требуется одна и та же обработка. Взять в пример вычисление какой-либо картинки на экране монитора: на сегодняшний день, разрешения экранов достигают более 2 миллионов пикселей, что представляет собой уже огромный пласт работы, который необходимо обрабатывать как можно быстрее. Производительность графического процессора достигается за использование параллелизма, при котором используется большое количество маленьких ядер. Поэтому, графические ускорители обладают большим количеством небольших ядер, тем самым позволяя работать с множеством данных, требующих одних и тех же инструкций за очень короткое время.

На рис. 15 можно увидеть упрощенную разницу архитектур CPU и GPU:

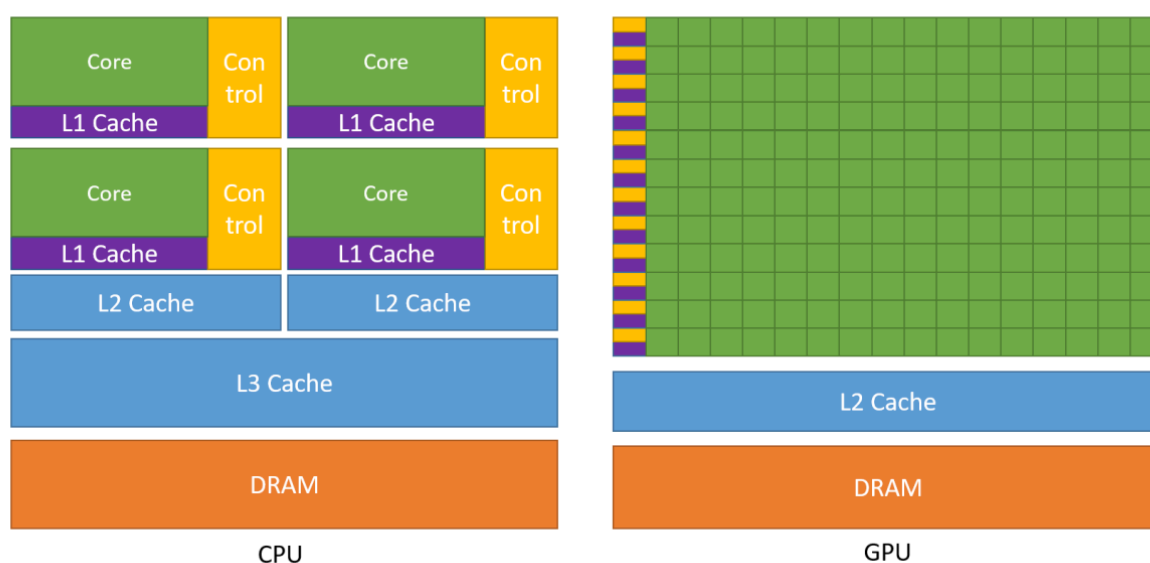


Рис. 15 — Сравнение архитектур GPU и CPU

2.2. Интерфейсы графического ядра

Графическое ядро - сложный блок, который содержит в себе большое количество логики. Для успешного взаимодействия ядра необходимо определиться с входными и выходными портами блока. Одним из способов задачи входных и выходных портов блока является составление интерфейсов блока.

Интерфейс - это совокупность средств, правил и инструментов, которые обеспечивают взаимодействие между двумя системами [21]. В аппаратном мире существует огромное количество интерфейсов созданные под разный спектр задач. При разработке графического ядра в первую очередь нужно основополагаться на производительность ядра. Следовательно, чем быстрее будет происходить взаимодействие с памятью для ядра, тем меньше будет простоя во всем ядре. Это означает, что для реализации интер-

фейса для передачи данных необходимо выбирать такие интерфейсы, которые обеспечат минимальную задержку или же быструю передачу данных.

Разработка собственного интерфейса не является целесообразной, так как добавляет проблем, которые необходимо нужно будет решать на этапе построения основы интерфейса. Даже при создании собственного высокоскоростного интерфейса необходимо также потратить время на написание документации и верификации разработанного блока. К тому же, блоки с интерфейсами собственной реализации сложно портируемы в проекты, так как зачастую в реальных аппаратных проектах используются уже существующие интерфейсы различного типа. По все вышеперечисленным причинам выбор был сделан в использовании уже существующих интерфейсов

Интерфейсы AMBA (Advanced Microcontroller Bus Architecture или продвинутая архитектура шины микроконтроллера) - семейство интерфейсов, разработанные компанией ARM, предназначены для соединения различных блоков, процессоров и периферии внутри систем на кристалле (СнК) и микросхем ПЛИС.

Одним из таких интерфейсов является интерфейс AMBA АНВ (Advanced High-performance Bus - продвинутая высокопроизводительная шина). Эта шина предназначена для высокоскоростных компонентов, таких как процессорные ядра, контроллеры памяти и прямой доступ к памяти (DMA). Данный интерфейс позволяет производить пакетную передачу, конвейеризацию операций и отдельные транзакции.

Также существует интерфейс под названием AMBA APB (Advanced Peripheral Bus - продвинутая периферийная шина) - предназначена для взаимодействия с низкоскоростной периферией, например, для взаимодействия с регистрами в периферии системы. Подобно АНВ, эта шина имеет фазы адреса и данных, но значительно урезанный несложный список сигналов. Интерфейс прост в разработке, но обладает недостаточной скоростью для передачи данных.

Последний среди трёх протоколов интерфейсов называется AMBA AXI (Advanced eXtensible Interface - продвинутый расширяемый интерфейс). Нацелен на высокопроизводительных и высокочастотных средств и включает возможности, которые делают интерфейс пригодным для высокоскоростных интерфейсов. Обладает отдельными фазами адреса/управления и данных, поддержкой передач невыровненных данных, применяя стробы байта, передач на основе пакетов, с выдачей лишь начального адреса, выдачей множества внешних адресов с неупорядоченными ответами и легкостью

добавления регистровых стадий для обеспечения близких задержек. Обладает очень сложной логикой и трудна в разработке [22].

Сравнительную таблицу всех представленных интерфейсов AMBA можно увидеть в табл. 1:

Таблица 1 — Сравнительная таблица интерфейсов AMBA

Интерфейс	APB	AHB	AXI
Производительность	Низкая	Высокая	Очень высокая
Сложность разработки	Низкая	Средняя	Высокая
Применение	Медленная периферия	Системная шина среднего уровня	Память, CPU, DSP, графика

2.2.1. Интерфейс передачи данных и инструкций (AHB)

Графическое ядро, само по себе, не генерирует данные, с которыми оно будет работать. Это значит, что в ядро необходимо загрузить данные, с которыми она будет работать. Также немало важно уделить внимание инструкциям, ведь инструкции также должны поступать в графическое ядро, для того чтобы ядро знало, что необходимо сделать с пришедшими данными. Существуют различные интерфейсы для передачи данных, которые предоставляют возможность обмениваться данными между аппаратными блоками.

Для первой итерации графического ядра для передачи данных и инструкций будет достаточно интерфейса AHB. В будущих итерациях стоит попробовать заменить интерфейс AHB и AXI и сравнить их влияние на производительность всего ядра графического процессора.

2.2.2. Интерфейс передачи данных с регистровой картой (APB)

Помимо передачи данных и инструкций в графическое ядро, ядро также должно выдавать какую-либо информацию о его состоянии. Один из способов - выводить все сигналы наружу. При таком подходе разработки портов графического ядра может возникнуть сложность с подключением: для сущностей, взаимодействующих с ядром (например обработчик прерываний), нет нужды отслеживать каждый такт состояния ядра. К тому же, в современных архитектурах графических процессоров количество ядер является очень большим и ядра образуют кластеры. Большое количество ядер с портами состояний усложняет подключение их всех к обработчикам ядер и отслежива-

нию состояний. Поэтому не стоит выводить всевозможные состояния ядра наружу - все данные можно хранить в специализированных регистрах. В данных регистрах можно сохранять статус ядра, включая его ошибки, состояние готовности. Помимо таких данных ядро может содержать в себе промежуточные значения для будущих вычислений. Данный набор структур называется регистровой картой и не только содержит в себе статусную информацию и промежуточные данные в ядре, но и управляющие сигналы. Как минимум, ядро должно каким-то образом запускаться, иметь возможность настроить собственный программный счётчик или отключить те или иные потоки в ядре. При грамотной разработке, в регистровой карте можно размечать отдельные регистры под разные спектры задач. Взаимодействие регистровой карты с внешними блоками должно также происходить через высокоскоростной протокол. Для общения с регистровой картой ядра достаточно разработать интерфейс APB для передачи данных и параметров ядра.

2.3. Поток

Графическое ядро должно содержать в себе вычислительные элементы, которые способны вычислять различные данные, приходящие в них. Элемент в графическом ядре, способный вычислять полученные данные будет в дальнейшем называть потоком (thread). Именно с помощью него, графика будет высчитываться различными математическими операциями. Количество потоков в ядре задаётся самим разработчиком. В различных архитектурах графических процессоров соотношение 1 поток к 1 ядру не применяется. Это вызвано тем, что объем данных, которые необходимо обработать слишком большой. Но при этом, часто возникает ситуация, когда над большим объемом данных необходимо произвести одну и ту же операцию. Операции, которые вызываются инструкциями, могут часто повторяться и поэтому в архитектурных решениях графических ядер зачастую у потока нет индивидуального декодера, ведь инструкция которая поступает в ядро является общей для всех потоков внутри ядра. Таким образом, большое количество потоков, исполняющие одну и ту же инструкцию, могут быстро выполнять вычисления над приходящими данными. Поэтому в одном графическом ядре используется много потоков, которые выполняют одну и ту же инструкцию над разными данными. Данное явление в разработке архитектуры называется SIMD.

2.3.1. Классификация параллельных вычислительных моделей (SIMT vs SIMD, MIMD).

Стоит понять концепцию и смысл выбора архитектуры параллелизма SIMD в архитектурах графических процессоров.

В разработке графических ускорителей принимают архитектурную классификацию параллелизма в потоках команд и данных SIMD, которая является одной из основных возможных архитектур параллелизма, основанных на таксономии Флинна. Среди основных классов наблюдаются следующие виды:

- ОКОД (SISD - single instruction, single data) - одиночный поток команд, одиночный поток данных;
- ОКМД (SIMD - single instruction, multiply data) - одиночный поток команд, множественный поток данных;
- МКОД (MISD - multiply instruction, single data) - множественный поток команд, одиночный поток данных;
- МКМД (MIMD - multiply instruction, multiply data) - множественный поток команд, множественный поток данных [23].

Все классы архитектур по Флинну представлены на рис. 16:

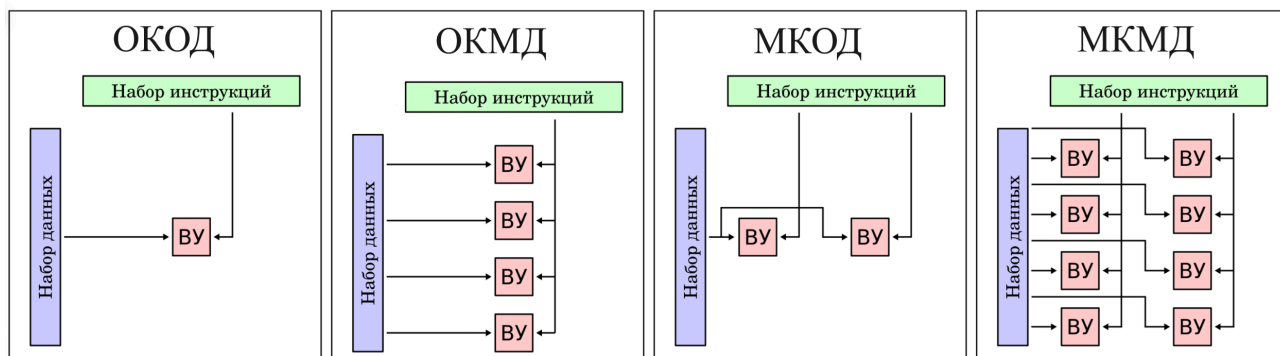


Рис. 16 — Все виды архитектур по таксономии Флинна

Примером архитектуры SISD является традиционный компьютер фон-Неймановской архитектуры с одним процессором, который выполняет последовательно одну инструкцию за другой, работая с одним потоком данных. SISD не является параллельной [24]. Основывать графический процессор на архитектуре SISD приведёт к слабой производительности, так как архитектура SISD вызывает ограничение скорости из-за своей природы. Первые процессоры основывались на этой архитектуре.

Архитектура SIMD позволяет обеспечить параллелизм на уровне данных, позволяя работать с большим количеством информации, над которой требуется работа

лишь одной конкретной инструкции. [25] Технология SIMD не только улучшает производительность вычислительных систем, но и уменьшает площадь, ибо количество необходимых шин, для того чтобы связать аппаратные блоки существенно уменьшается. На основе этой архитектуры была разработана архитектура SIMT (Single Instruction Multiple Threads) - одна инструкция, множество потоков.

Архитектура MISD основана на том, что множество инструкций будет работать с одиночным потоком данных. Это может быть полезно в различных системах с отказоустойчивостью: в космонавтике или авионике. Множество процессоров работают над одним и тем же потоком данных. [26] В нейросетях и графике нет необходимости различными инструкциями выстраиваться в очередь для обработки одного потока данных. При такой архитектуре графическое ядро будет обладать большими задержками, что противоречит цели в создании производительной архитектуры графического ускорителя.

MIMD архитектура является распространенной архитектурой в компьютерах с несколькими процессорами. Благодаря такой архитектуре, можно обеспечить эффективное использование аппаратных ресурсов. При этом, ядра, которые будут использовать архитектуру MIMD будут сложными, т.к. на каждый поток команд и инструкций необходимо разработать блок выборки и декодирования команд, что увеличит место на кристалле, а также увеличит объем потребляемой энергии. Из-за такой архитектуры также необходимо разработать сложную систему планирования, потому что в MIMD могут возникнуть проблемы с взаимной блокировкой и состязанием за обладанием ресурсами. Это связано с тем, что потоки, пытаясь получить доступ к ресурсам, могут встретиться непредсказуемым способом на одном и том же ресурсе и такую проблему необходимо будет решать отдельными компонентами в архитектуре, которые также будут занимать место и энергию на кристалле [27].

Среди всех архитектур для разработки графического ядра явно подходит архитектура SIMD. На её основе будет разрабатываться архитектура графического ядра.

2.3.2. Структура потока

Каждый поток в любом графическом ядре имеет одну и ту же структуру. Разработав один поток, можно масштабировать и конфигурировать ядро, в зависимости от нужд и поставленных задач для самого ядра. В ядро приходит набор данных и инструкций, которые необходимы потоку, для выполнения обработки математическими

операциями над пришедшими данными. Также немало важен компонент, который будет производить вычисления.

2.3.2.1. Арифметически-логическое устройство

Таким компонентом в аппаратном разрабтке является функциональный блок, арифметическо-логическое устройство (АЛУ или ALU). Оно способно производить операции над операндами и выводить результат. АЛУ крайне необходимы в вычислениях и очень востребованы в вычислительных ядрах.

Обобщенную схему арифметически-логического устройства можно увидеть на рис. 17:

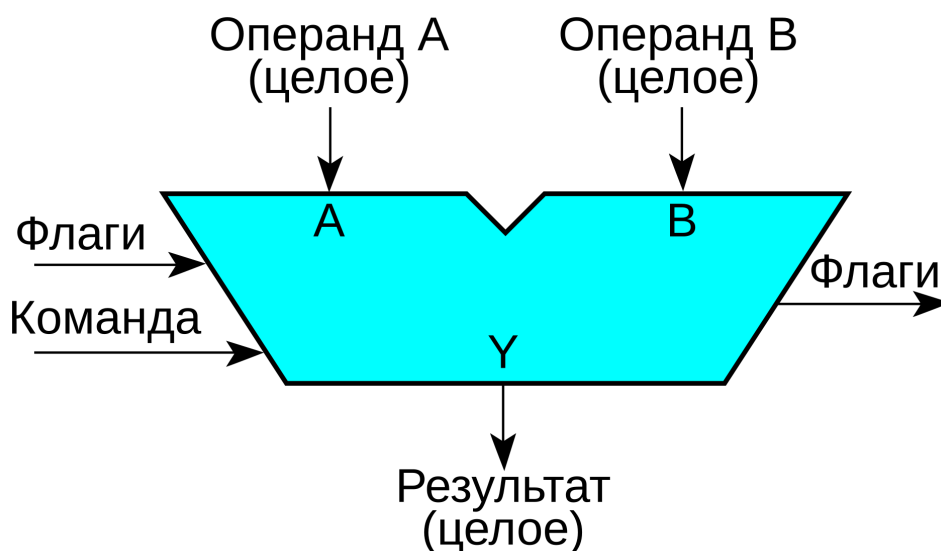


Рис. 17 — Обобщенная схема арифметически-логического устройства

Типичное АЛУ может выполнять сложение, вычитание, сравнение значений, операции И или ИЛИ. АЛУ может иметь различные конфигурации и возможности, в зависимости от поставленных требований. В основном, АЛУ принимает на вход 2 или 3 операнда (данные), над которыми будут производиться вычисления, опкод или операция, которая будет производиться над операндами. В продвинутых АЛУ могут присутствовать входящие статусы, например, для определения формата числа, а также выходящие статусы для информирования управляющим блоком о возникших ситуациях в АЛУ [28].

В графических процессорах могут находиться большое количество арифметическо-логических устройств, которые одновременно работают над параллельными данными. АЛУ в потоке является «мозгом» потока и критически важен в реализации графического процессора [29]. Но, стоит подметить, что АЛУ в своей типичной реализации работает с целочисленными числами, что подходит для индексирования и адресования в потоках. Поэтому в потоке будет АЛУ, который будет заниматься

вычислением адресов и индексирования, но для вычисления самой графики необходимо схожий, но отличный компонент от АЛУ.

2.3.2.2. Математический сопроцессор

Основные типы чисел, которые используются в написании шейдеров являются вещественные числа. С помощью вещественных чисел можно добиться необходимой точности в отрисовке графики. Один из основных языков для программирования шейдеров, HLSL строго требует использования вещественных чисел. Поэтому для вычисления вершин и пикселей в потоке необходимо наличия специального компонента, способного работать с вещественными числами [30].

Арифметически-логическое устройство не способно работать с вещественными данными. Для вычисления данных с плавающей точкой существуют математические сопроцессоры (FPU), которые позволяют осуществить выполнение математических операций над вещественными числами.

Целочисленные значения представлены как набор степени двойки с отдельным выделением бита знака, если требуется хранить знаковое число. С хранением вещественных чисел все несколько сложнее. Числа со знаком хранятся по определенному техническому стандарту, с помощью которого достигается унификация математических операций во всех современных процессорах и языках программирования [31].

Схему хранения чисел с плавающей точкой формата IEEE 754 можно увидеть на рис. 18:

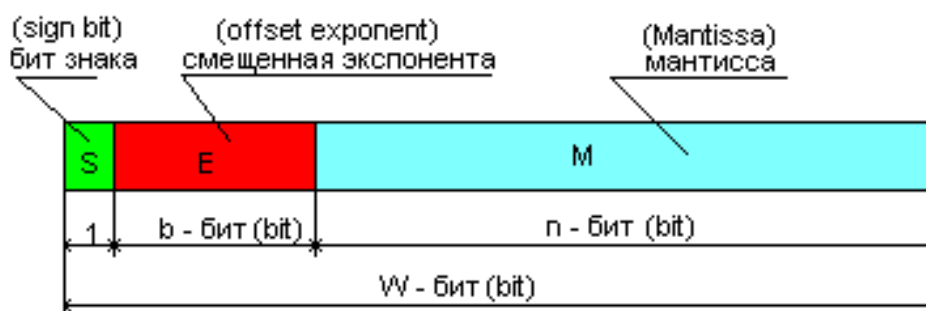


Рис. 18 — Схема хранения чисел с плавающей точкой по стандарту IEEE 754

Ячейка для хранения чисел по техническому стандарту IEEE 754 состоит из трёх полей:

- Бита знака (S) - С помощью этого бита определяется положительность или отрицательность числа. Если все

биты являются нулями, то число в ячейке равно 0, но если бит знака будет установлен в 1, то значение 0 приобретёт отрицательный знак;

- Смещенная экспонента (E) - Является экспонентой 10-ки. С помощью экспоненты можно двигать запятую

в мантиссе, позволяя получить необходимое число. От этого числа вычитается смещение, для получения нужной степени для возведения десятки и перемножения с мантисой;

- Мантисса (M) - Дробная часть числа, которая определяет его точность.

Размер полей смещенной экспоненты и мантисы может быть разным. В зависимости от разрядности самого числа формируются количество битов для мантисы и экспоненты [32], как показано на табл. 2:

Таблица 2 — Размер полей в зависимости от разрядности числа

Количество битов	16	32	64	128
Размер бита знака	1	1	1	1
Смещенная экспонента	5	8	11	15
Мантисса	10	23	52	112

IEEE 754 не только определяет порядок хранения не только чисел но и служебных состояний, таких как:

- бесконечности
- NaN (Not a Number) - не является числом
- Ноль (+0 и -0)

Также в стандарте описаны виды округлений, которые должны поддерживаться для соблюдения стандарта. Среди стандарта также присутствуют статусные флаги, которые информируют о различных ошибках, возникших при выполнении операции над операндами. К ним относятся

- Недопустимые операции (NV) - происходит, когда математическая операция не имеет математически определенного

смысла. Это происходит при попытке извлечения квадратного корня из отрицательного числа или при попытке взаимодействия с нечисловым значением (NaN). Результату присваивается NaN;

- Деление на ноль (DZ) - происходит при попытке деления на ноль. Результату присваивается бесконечность;
- Переполнение (OF) - происходит при получении результата, который нельзя поместить в отведенный для числа

объем памяти. Результат заменяется бесконечностью;

- Потеря точности (UF) - происходит при получении результата, который нельзя представить в указанном формате с указанной точностью;
- Неточность (NX) - возникает практически при любых операциях с округлением, когда точный Математический результат невозможно записать в виде конечного числа бит (Например, когда 10 делится на 3).

На основе стандарта хранения вещественных чисел IEEE 754 можно реализовывать FPU, которая способна работать с вещественными числами. Аппаратная разработка FPU является нетривиальной, поэтому в качестве математического сопроцессора будет использована уже существующая реализация в открытом доступе. Данный мультипроцессор поддерживает операции:

- сложения;
- умножения;
- совмещенного сложения и умножения;
- деления;
- извлечения корня;
- нахождения минимума или максимума;
- сравнения;
- извлечения знака;
- перевод чисел в различные форматы, как целочисленные, так и вещественные [33].

Этого набора операций достаточно для использования FPU в потоке.

2.3.2.3. Регистровый файл

FPU не может хранить в себе значения, которые ей нужны для вычислений. При этом значения которые вычислил FPU также нужно где-то хранить, ведь они могут использоваться в будущих вычислениях или потребуются для внешних компонентов позже. Нет смысла держать данные для FPU, поэтому для этого используют память.

Память - это устройство для упорядоченного хранения и выдачи информации. Различные запоминающие устройства отличаются способом и организацией хранения данных. Для каждой памяти определяется её размер:

- количество ячеек в памяти, в которые можно записать или считать данные;
- разрядность каждой ячейки в памяти [34];

Ячейкой для хранения значения в памяти называют регистром. Из регистров составляются блоки регистровой памяти, которые в свою очередь называются регистровыми файлами (Register file). Регистровая память в потоке необходима для хранения промежуточных данных и константных значений, для их обработки. В регистровую память можно записывать или читать с нее. Все компоненты потока, а также некоторые компоненты ядра могут взаимодействовать с регистровым файлом. Для создания регистрового файла, помимо самих регистров, необходимо создать порты для возможности чтения и записи регистров.

Для воспроизведения чтения с регистрового файла, необходимо знать откуда читать, поэтому в каждом порте чтения будет сигнал адреса, который будет сигнализировать с какого адреса нужно выложить данные в порт чтения.

Для воспроизведения записи с регистрового файла, необходимо знать куда писать, данные, которые надо записать в регистр по адресу и сигнал включения записи, чтобы сигнализировать о валидности записи. Если сигнал включения записи выключен, независимо от того, что будет передано по сигналу записи и адреса, в регистры регистрового файла ничего записано не будет.

Первым делом, точно известно, что для разработки потока необходим как минимум 1 порт для чтения из регистрового файла и 1 порт для записи в регистровый файл. Количество портов для чтения можно указать произвольное количество, так как конфликтов чтения в регистровом файле потока не может произойти. Но повышение количества портов для чтения избыточно и громоздко, поэтому необходимо определиться с конкретным числом портов чтения. В то же время выделить дополнительные порты для записи является проблематичной задачей, так возникает проблема конфликта записи, которую необходимо будет решать.

Так как FPU содержит возможности операции совмещенного сложения и умножения для него необходимо как минимум 3 порта на чтение, чтобы принять все необходимые операнды для вычисления. Для записи результатов FPU потребуется один порт на запись. Поэтому со стороны FPU уже есть 3 порта для чтения. Вывод результата FPU стоит пропускать через мультиплексор (MUX), который будет сохранять данные в регистровый файл, в зависимости от управляющего сигнала мультиплексора. Мультиплексор необходим, чтобы указать приоритет для компонентов, которые будут записывать данные через единственный порт записи. Это исключит конфликт записи, ведь записывать данные сможет лишь один блок.

В графическом ядре будет находиться специальный блок, с помощью которого можно загружать и выгружать данные из регистрового файла потока. Данному блоку необходимо два порта для чтения и один для записи. Два порта для чтения будут выделены из регистрового файла специально под этот блок, а порт записи будет разделен совместно с FPU через MUX.

Результируя, можно подсчитать, что для регистрового файла необходимо 5 портов для чтения данных: 3 для FPU, 2 для блока загрузки и выгрузки, а также 1 порт для записи, который будет общим для всех компонентов, которые могут записывать в регистровый файл.

В качестве размера регистрового файла было принято взять 32 - минимальный размер для запуска ядра, где два регистра будут выделены под константные значения. Самый первый регистр будет заполнен нулем, чтобы ускорить инициализацию регистрового файла нулями, с помощью инструкций. Второй константный регистр будет находиться в конце и определять номер потока. В эти два регистра нельзя записать значения напрямую. Нулевой регистр всегда будет 0, а номер потока, сможет выдаваться через регистровую карту ядра. Понижение размера регистрового файла не является возможным, т.к. количество регистров для сохранения локальных данных для вычислений будет является недостаточным для реализации современной графики. Увеличение количества регистрового файла позволит увеличить количество возможных данных для хранения, а также уменьшит количество инструкций для сохранения необходимых значений, но при этом значительно увеличит площадь регистрового файла и следовательно - размер одного потока.

Разрядность регистровой карты для первой итерации графического ядра стоит взять, как 32 битные. Конечно, большая разрядность позволяет обеспечить большую точность вычислений графики, но при этом, повышение разрядности регистров может увеличить площадь одного потока, что уменьшит общее количество потоков, которые можно будет расположить на условной единице площади.

2.3.2.4. Архитектура системы

Имеющиеся компоненты в потоке описанные ранее, должны принимать на вход управляющие сигналы. Для блока:

- FPU необходимо указать тип операции, которые будут производиться над данными и тип округления, который будет применяться;

- ALU необходимо указать только тип операции;
- Регистровому файлу необходимо указать адреса в портах вывода и управлять сигналами записи, такими как
адрес записи и сигнал разрешения записи.

Всем перечисленным блокам, нужна инструкция, с помощью которой ими можно управлять и взаимодействовать. Для управления потоком необходимо разработать набор инструкций, по которому можно взаимодействовать с потоком. Это означает, что необходимо создать архитектуру системы команд (Instruction Set Architecture, ISA).

ISA - это абстрактная модель, определяющая программируемый интерфейс процессора и описывающая способы взаимодействия между программными и аппаратными элементами между собой. Архитектура системы команд определяет систему команд, типы данных, регистры, а также программируемый интерфейс для управления основной памятью, включая режимы адресации, механизмы виртуальной памяти и модели согласованности памяти. При необходимости ISA можно расширять путём добавления новых команд или дополнительных возможностей с сохранением старого функционала [35].

2.3.2.4.1. Архитектура набора инструкций RISC-V

Для разработки собственной архитектуры набора команд, необходимо посмотреть на уже существующие архитектуры. Одна из них является архитектура RISC-V, которая использует открытый набор команд (ISA), построенный на принципах RISC (Reduced Instruction Set Computer):

- Простота операций - инструкции могут выполняться за 1 такт;
- Операции типа «регистр-регистр» - основные вычисления производятся только внутри памяти процессора, а именно в
регистрах. Прямые операции изменения данных в оперативной памяти запрещены;
- Упрощенная адресация - используется небольшое количество режимов адресации, что ускоряет формирование адресов
для памяти;
- Большое количество регистров - благодаря наличию множества регистров общего назначения, значительно снижает
необходимость частого обращения к медленной памяти;
- Инструкции фиксированной длины - все машинные инструкции имеют одинаковый размер в битах, что упрощает декодировку

и ускоряет работу процессоров [36].

Базовые инструкции RISC-V обладают фиксированной длиной - 32 бита. В этот размер необходимо вместить всю необходимую информацию для выполнения необходимой задачи. Для разных операций требуются разные параметры (регистры, смещения в памяти, прямые значения или константы), поэтому архитектура RISC-V обладает разными форматами. В архитектуре набора команд существует 6 основных форматов:

- R (Register): для операций «регистр-регистр». В данном типе инструкций должны находиться адреса 3 регистров (два регистра источника и один приёмник);
- I (Immediate): для операций с константами и загрузки данных из памяти. Данный формат инструкций содержит в себе два адреса регистра и 12-битное число для его использования в вычислениях;
- S (Store): для записи данных в память. Содержит два адреса регистра и 12-битную константу, для задачи смещения;
- B (Branch): для осуществления условных переходов. Схожий с форматом S, но слегка видоизменен для вычисления адресов переходов;
- U (Upper Immediate): через другие инструкции можно записать лишь 12-битную константу в регистр. Так как регистры являются 32-битными, для записи 20 старших битов в регистр используется данный формат инструкций;
- J (Jump - прыжок): для безусловных переходов, содержит в себе 20-битное смещение.

Форматы 32-битных инструкций RISC-V изображены на

32-bit RISC-V Instruction Formats																																
Instruction Formats	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Register/register	funct7							rs2					rs1					funct3			rd				opcode							
Immediate	imm[11:0]												rs1				funct3			rd				opcode								
Upper Immediate	imm[31:12]																				rd				opcode							
Store	imm[11:5]							rs2					rs1				funct3			imm[4:0]				opcode								
Branch	[12]	imm[10:5]							rs2					rs1				funct3			imm[4:1]			[11]	opcode							
Jump	[20]	imm[10:1]										[11]	imm[19:12]								rd				opcode							
● opcode (7 bit): partially specifies which of the 6 types of <i>instruction formats</i> ● funct7 + funct3 (10 bit): combined with opcode, these two fields describe what operation to perform ● rs1 (5 bit): specifies register containing first operand ● rs2 (5 bit): specifies second register operand ● rd (5 bit):: Destination register specifies register which will receive result of computation																																

Рис. 19 — Форматы 32-битных инструкций RISC-V

RISC-V имеет модульный принцип. Набор команд архитектуры набора инструкций RISC-V состоит из фиксированной базовой части и стандартных или пользовательских расширений, которые добавляют поддержку вычислений с плавающей точкой или векторных операций. Например, при необходимости, существует набор сжатых команд до 16 бит - для микросхем или расширение для умножения и деления целых чисел. Разрядность инструкций может изменяться в зависимости от поставленных задач.

2.3.2.4.2. Архитектура набора инструкций графического ядра

Основываясь на архитектуре набора инструкций RISC-V можно составить собственный набор инструкций основываясь на создании форматов инструкций. Для первой итерации необходим базовый минимум для реализации инструкций, поэтому изначально будут реализованы критически необходимые инструкции. Среди таких инструкций, необходимых в первую очередь, стоит реализовать инструкции загрузки и сохранения данных, сохранения константных значений, инструкции управления FPU, а также управления ALU.

Первую итерацию архитектуры набора инструкций для управления потоком, можно реализовать с помощью трёх форматов инструкций:

- F (FPU) - формат инструкций для управления математическим сопроцессором.

Формат должен в себе содержать 3 адреса регистров для

передачи их значений как операндов, адрес регистра в который будет сохранено значение, тип операции и способ округления полученного значения;

- L (Load-Storage Unit) - формат инструкций для управления загрузкой и сохранением данных. Также с помощью данного формата можно

осуществлять вычисления значений через ALU, например, для получения адресов и смещений. Должен содержать в себе 2 адреса регистра источников, адрес регистра в который будет записываться результат, а также константное значение для задачи смещения при загрузке или сохранении из внешней памяти;

- U (Upper Immediate) - формат для загрузки старших битов регистра. Обязательное условие инструкции заключается в необходимости поля

константного значения, превышающую разницу размера регистра и количества бит поля константного значения инструкции типа L.

Все инструкции будут иметь размер в 32 бита. Следовательно, необходимо разместить все поля необходимых значений для инструкции. В качестве определения типа инструкции будет выделено 2 бита. Следующие 5 битов будут определены для опреде-

ления операции в каждом типе инструкции. По своей сути, эти 7 битов будут определять тип операции и саму операцию в инструкции и называются полем opcode. Данное поле будет находится у всех возможных форматов инструкций.

В форматах инструкций требуются адреса регистров. Сами регистры будут использоваться из регистрового файла. Регистры будут именоваться, как rsX (register source) - регистр источник, где X - номер источника, и rd (register destination), как регистр, куда будут сохраняться значения. Зная, что размер регистрового файла составляет 32 регистра, можно задать минимально необходимую длину адреса в битах. Для регистрового файла, размером 32 бита необходимо выделить $\log_2(32) = 5$ битов для адреса регистра. Для форматов инструкций:

- F-типа необходимо 5 битов для rd регистра и по 5 битов для rs1, rs2 и rs3 (Итого 20 битов для адресов);
- L-типа необходимо 5 битов для rd регистра и 5 битов для rs1 и rs2 (Итого 15 битов для адресов);
- U-типа необходимо 5 битов для rd регистра (Итого 5 битов для адресов).

Для F формата инструкций необходимо также выделить 3 бита для задачи типа округления при вычисления значений. Все оставшиеся биты у инструкций будут выделены под константный значения.

Содержания инструкций каждого формата показано на рис. 20:

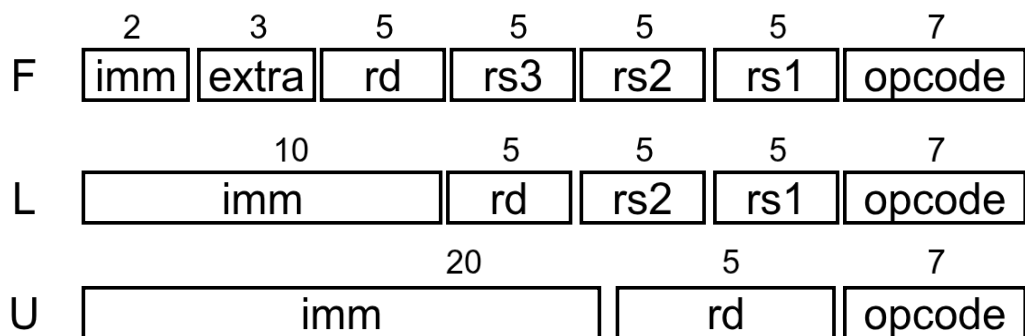


Рис. 20 — Форматы 32-битных инструкций для графического ядра

Такая архитектура формата инструкций позволит создавать первые необходимые инструкции для управления не только потоком, но и ядром.

2.3.2.5. Общая структура потока

Все ранее описанные компоненты формируют основу для реализации потока графического ядра. При разработке архитектуры потока также может понадобится вспомогательные блоки, которые организуют корректное взаимодействие компонентов

блока между собой. При этом сам по себе поток представляет компонент ядра, который может выполнять

На рис. 21, представленная упрощенная схема блока потока:

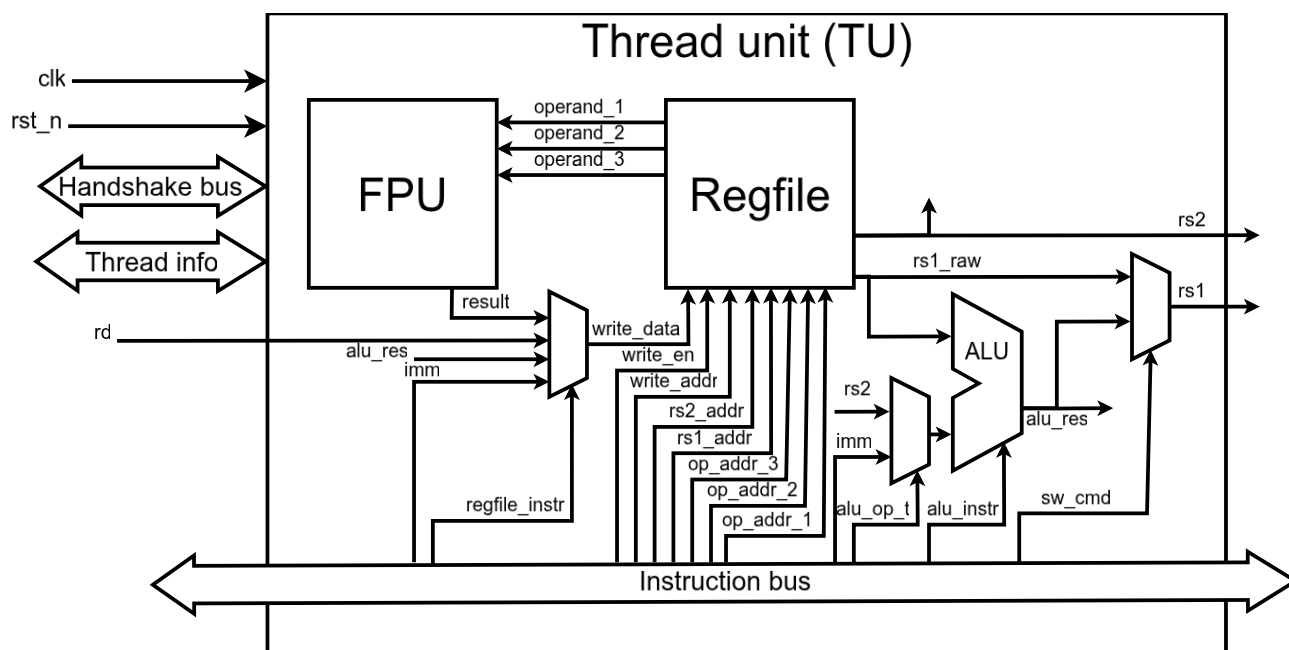


Рис. 21 — Упрощенная схема блока потока (Thread Unit)

2.3.3. Структура ядра

Заключение

Литература

- [1] И.А. Дудкин, Д.М. Старухин, и В.В. Черкасов, «Обзор и анализ 2D графических ускорителей».
- [2] И.А. Дудкин, Д.М. Старухин, и В.В. Черкасов, «Разработка архитектуры аппаратной реализации 2D графического ускорителя».
- [3] «Видеокарта». [Онлайн]. Доступно на: <https://ru.wikipedia.org/wiki/%D0%92%D0%B8%D0%B4%D0%B5%D0%BE%D0%BA%D0%B0%D1%80%D1%82%D0%B0>
- [4] «Nvidia». [Онлайн]. Доступно на: <https://ru.wikipedia.org/wiki/Nvidia>
- [5] «NVIDIA NV30». [Онлайн]. Доступно на: <https://www.techpowerup.com/gpu-specs/nvidia-nv30.g20>
- [6] Э. С. Ершов и Е. А. Сухотерин, «Архитектура графических процессоров Nvidia восьмого поколения с точки зрения вычислений общего назначения».
- [7] «NVIDIA Tesla A Unified Graphics and Computing Architecture». [Онлайн]. Доступно на: <https://www.techpowerup.com/gpu-specs/docs/nvidia-tesla-architecture.pdf>
- [8] «NVIDIA's Next Generation CUDA Compute Architecture». [Онлайн]. Доступно на: <https://www.techpowerup.com/gpu-specs/docs/nvidia-fermi-compute-architecture.pdf>
- [9] «Turing (микроархитектура)». [Онлайн]. Доступно на: [https://en.wikipedia.org/wiki/Turing_\(microarchitecture\)](https://en.wikipedia.org/wiki/Turing_(microarchitecture))
- [10] «NVIDIA TURING GPU ARCHITECTURE». [Онлайн]. Доступно на: <https://www.techpowerup.com/gpu-specs/docs/nvidia-turing-architecture.pdf>
- [11] «amd». [Онлайн]. Доступно на: <https://en.wikipedia.org/wiki/AMD>
- [12] «Part I The first fully programmable GPUs-a review». [Онлайн]. Доступно на: <https://www.jonpeddie.com/news/part-i-the-first-fully-programmable-gpus-a-review/>
- [13] «История развития графики AMD/ATI. Часть 2 путь универсальности». [Онлайн]. Доступно на: https://club.dns-shop.ru/blog/t-99-videokartyi/101511-istoriya-razvitiya-grafiki-amd-ati-chast-2-put-universalnosti/#sub_Radeon__HD2000__DirectX__10__i__superskalyarnaya__TeraScale
- [14] «R600-Family Instruction Set Architecture». [Онлайн]. Доступно на: <https://www.techpowerup.com/gpu-specs/docs/ati-r600-isa.pdf>
- [15] «Evergreen Family Instruction Set Architecture Instructions and Microcode». [Онлайн]. Доступно на: <https://www.techpowerup.com/gpu-specs/docs/ati-evergreen-isa.pdf>
- [16] «Введение в OpenCL». [Онлайн]. Доступно на: <https://habr.com/ru/articles/124925/>

- [17] «Учимся разрабатывать для GPU на примере операции GEMM». [Онлайн]. Доступно на: <https://habr.com/ru/companies/yadro/articles/934878/>
- [18] «AMD GrAPhICS CORES NEXT (GCN) ARCHITECTURE». [Онлайн]. Доступно на: <https://www.techpowerup.com/gpu-specs/docs/amd-gcn1-architecture.pdf>
- [19] «Compute unit». [Онлайн]. Доступно на: <https://rocm.docs.amd.com/projects/omniperf/en/amd-staging/conceptual/compute-unit.html>
- [20] «RDNA (microarchitecture)». [Онлайн]. Доступно на: [https://en.wikipedia.org/wiki/RDNA_\(microarchitecture\)](https://en.wikipedia.org/wiki/RDNA_(microarchitecture))
- [21] «Интерфейс». [Онлайн]. Доступно на: <https://ru.wikipedia.org/wiki/%D0%98%D0%BD%D1%82%D0%B5%D1%80%D1%84%D0%B5%D0%B9%D1%81>
- [22] «АМБА». [Онлайн]. Доступно на: https://ru.wikipedia.org/wiki/Advanced_Microcontroller_Bus_Architecture
- [23] «Таксономия Флинна». [Онлайн]. Доступно на: https://en.wikipedia.org/wiki/Flynn%27s_taxonomy
- [24] «SISD». [Онлайн]. Доступно на: <https://ru.wikipedia.org/wiki/SISD>
- [25] «SIMD». [Онлайн]. Доступно на: <https://ru.wikipedia.org/wiki/SIMD>
- [26] «MISD». [Онлайн]. Доступно на: <https://ru.wikipedia.org/wiki/MISD>
- [27] «MIMD». [Онлайн]. Доступно на: <https://ru.wikipedia.org/wiki/MIMD>
- [28] «Арифметико-логическое устройство», в *Цифровая схемотехника и архитектура компьютера: RISC-V*, ДМК Пресс, chap. 5.2.4, сс. 304–309.
- [29] «ALU». [Онлайн]. Доступно на: https://en.wikipedia.org/wiki/Arithmetic_logic_unit
- [30] «Числа с плавающей точкой», в *Цифровая схемотехника и архитектура компьютера: RISC-V*, ДМК Пресс, chap. 5.3.2, сс. 315–316.
- [31] «IEEE_754». [Онлайн]. Доступно на: https://ru.wikipedia.org/wiki/IEEE_754
- [32] Microprocessor Standards Committee, «IEEE Standard for Floating-Point Arithmetic».
- [33] *FPnew - New Floating-Point Unit with Transprecision Capabilities*. [Онлайн]. Доступно на: <https://github.com/openhwgroup/cvfrpu>
- [34] «Лабораторная работа №3 "Регистровый файл и память инструкций"», в *Архитектуры процессорных систем*, АО "РИЦ "Техносфера", сс. 87–90.
- [35] «Instruction Set Architecture». [Онлайн]. Доступно на: https://en.wikipedia.org/wiki/Instruction_set_architecture
- [36] «RISC Principles», в *Guide to RISC Processors for Programmers and Engineers*, Springer, 2005, chap. 3, сс. 39–44.